AIDE

Adaptive Intelligent Dynamic Embedding — project overview

CD753PC Project, MGIT Hyderabad · Version 0.2 · October 2026

In one sentence: AIDE hides an encrypted message inside an ordinary photograph so that it looks unchanged, survives compression, noise and cropping on the way, can be proven untampered on arrival — and it also ships the tools an analyst needs to detect hidden data in intercepted images.

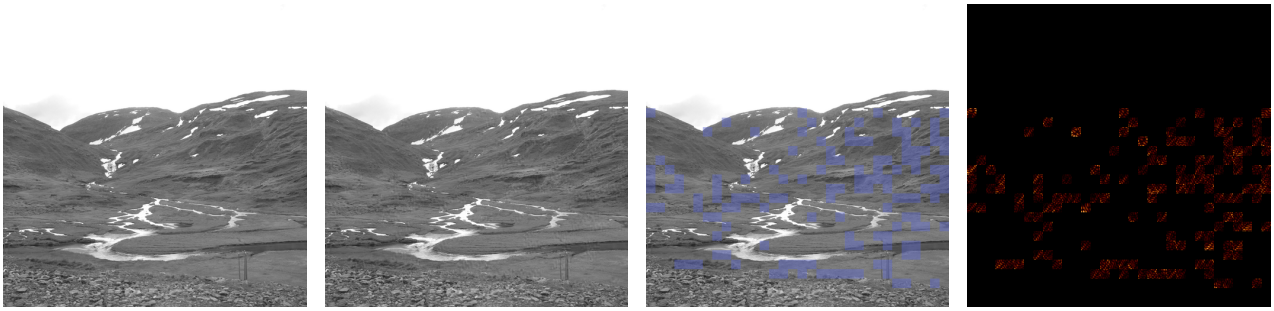
1. What we built

AIDE is a complete, working steganography system — not a notebook experiment. It covers both sides of the defence and intelligence use case from the project report:

- **Operator side (hide):** encrypt a message with AES-256-GCM, protect it with Reed-Solomon error correction, and embed it into the image blocks a trained AI model picks as safest. The receiver recovers the exact message or gets a clear error — never a silently corrupted one.
- **Analyst side (detect):** screen suspicious images with no key, using chi-square, Sample Pairs and RS analysis plus a trained machine-learning steganalyser, with a spatial suspicion heatmap and batch CSV screening.
- **Tamper evidence:** per-block SHA-256 hashes and histogram tests mark exactly which regions were altered, and those regions are fed to the error-correction decoder as erasures.
- **Four robustness profiles** (quality, balanced, robust, fortress) and two modes: *sidecar* (small JSON map travels with the image) and *blind* (only the image and the shared secret are needed).
- **Three ways to use it:** the `aide` command-line tool, a web console (Operator / Analyst / Model Insights), and a versioned REST API with an OpenAPI contract.

How a message travels through AIDE

Step	What happens
1. Analyse	Image split into blocks; texture, entropy, variance and edge strength measured for each
2. Decide	Trained model scores each block S(b) and picks method (DCT/DWT/LSB), bits per block, or skip
3. Encrypt	AES-256-GCM with a key derived by scrypt from the shared secret (and the image)
4. Protect	Reed-Solomon parity, shortened codewords and interleaving spread damage across codewords
5. Embed	Bits written as DCT coefficient-pair relations (QIM) in the chosen blocks
6. Transmit	Image may be JPEG-compressed, resized, noised, filtered or cropped on the way
7. Recover	Tamper map finds damaged blocks; GMD decoding repairs bytes; GCM tag proves the message is exact



Original cover | Stego (message hidden) | Blocks AIDE chose | Change map (x10)

Real output from this repository: a 76-byte message hidden in a BOSSbase photo with the balanced profile (PSNR 42.0 dB) and recovered bit-exact.

2. Technologies

Area	Technology
Language / runtime	Python 3.10-3.12, CPU-only (no GPU or CUDA needed)
Image processing	OpenCV, NumPy, SciPy, Pillow, scikit-image, PyWavelets (DWT)
Machine learning	scikit-learn (decision tree / tiny MLP, gradient-boosted trees), joblib
Cryptography	pycryptodome: AES-256-GCM authenticated encryption, scrypt key derivation
Error correction	reedsolo: Reed-Solomon codes, interleaving, GMD reliability decoding
Web + API	Flask console, versioned REST API with OpenAPI spec, gunicorn, API-key auth
Engineering	pytest (146 tests), ruff lint, GitHub Actions CI, Docker, Makefile
Dataset	BOSSbase 1.01 real photographs (334 images, 342k blocks for training)

Core techniques

- **Transform-domain embedding:** DCT mid-frequency coefficient-pair quantisation index modulation (QIM), 2-level Haar DWT, and LSB as a fallback for smooth blocks.
- **Learned block scoring:** $S(b) = \alpha \cdot \text{Texture} + \beta \cdot \text{Entropy} + \gamma \cdot \text{Variance} + \delta \cdot \text{Edge}$. The weights are learned, not hand-set: $\alpha = 0.55$, $\delta = 0.44$, entropy and variance near zero.
- **Security:** scrypt (memory-hard) key derivation, authenticated encryption, keyed pseudo-random block layout in blind mode.
- **Steganalysis features:** SPAM (subtractive pixel adjacency matrix) and DCT pair-gap histograms for the ML detector.

Architecture at a glance

Module	Responsibility
features.py / scoring.py / model.py	Block features, S(b) scoring, trained decision engine
crypto.py / ecc.py	AES-256-GCM + scrypt; Reed-Solomon, interleaving, GMD
embedding/ (lsb, dct, dwt)	Embed and extract (hard and soft decisions)
pipeline.py / blind.py	Sidecar and blind end-to-end embed / recover
tamper.py / attacks.py / metrics.py	Tamper map; 8 attack simulations; PSNR, SSIM, BER
steganalysis.py / quantitative.py / mlsteg.py	Classical, SPA/RS and ML detection
webapp/app.py, cli.py	Web console + REST API; aide command line

3. What we are supposed to do with this project

AIDE has three audiences. Each one uses it differently:

Who	Goal	How they use AIDE
Field operator / secure communicator	Send a confidential message through channels that recompress or damage images, without drawing attention	Pick a profile, check capacity, embed with a shared secret, send the image; receiver extracts with the same secret
Intelligence / forensic analyst	Find out whether intercepted media carries hidden data and roughly how much	Analyst tab or aide detect: one verdict per image, payload estimate, heatmap, batch CSV for whole folders
Student / researcher	Learn and evaluate modern steganography and steganalysis with honest measurements	Run the benchmark, compare profiles, retrain models, read Model Insights, extend modules

Typical workflow (demo)

- Start the console: `python webapp/app.py` and open http://127.0.0.1:5050.
- **Operator tab:** upload a cover, type a message and secret, embed. Then simulate an attack (JPEG, noise, crop...), run the tamper check and recover the message.
- **Analyst tab:** upload the stego image and see what each detector family says about it.
- **Model Insights:** show the learned weights, the robustness matrix and the detector evaluation.

Robustness profiles (measured on 30 BOSSbase covers)

Profile	PSNR	SSIM	Built for
quality	43.4 dB	0.984	Lossless channels, JPEG >= Q50
balanced (default)	40.1 dB	0.967	JPEG >= Q30, mild noise, filtering, rescaling
robust	35.0 dB	0.912	Noise s=20, salt & pepper, 10% crop (objective O4)
fortress	30.5 dB	0.803	Hostile channels, short messages



Bit-exact message recovery after each attack, by profile. Stronger profiles trade image quality for survival.

4. What we are predicting

AIDE never tries to predict or guess the hidden message — that is protected by AES-256-GCM. The machine-learning parts predict properties of **images and image blocks**:

Model	Input	What it predicts	Measured result
Decision engine (decision tree or MLP)	4 features per image block: texture, entropy, variance, edge	Which embedding method suits the block (LSB / DCT / DWT), how many bits it can safely carry (bpp), or SKIP	87.7% validation accuracy; bpp error 0.14 bpp (MAE)
ML steganalyser (gradient-boosted trees)	126 features per image: SPAM pixel-adjacency + DCT pair-gap histograms	Probability that an image hides an AIDE payload (clean vs stego)	AUC 0.999 on 100 held-out covers, 1% false positives
SPA / RS analysis (statistical estimators)	Pixel-pair statistics of one image	Estimated embedding rate, i.e. roughly how much data is hidden (LSB family)	Validated against ground truth
Chi-square PoV (statistical test)	Pixel value histogram	Yes / no: sequential LSB embedding present	Classical baseline

How the decision engine is trained

- **Method labels** come from the report's rule (low texture → LSB, strong edges → DWT, otherwise DCT) with thresholds at the 33rd/67th percentiles, so classes are balanced. The model learns to reproduce and smooth this rule from all four features.
- **Payload labels are measured:** random bits are really embedded at 4/3/2/1 bpp and block PSNR is measured; the label is the highest safe bpp, or SKIP. Target block quality: 42 dB.
- Trained on 334 BOSSbase images (342,016 blocks). Inference costs about 0.16 ms per block on one CPU core; the model file is 13 KB.

Honest finding on detectability

> AIDE images pass all classical blind detectors (chi-square, SPA, RS: 0/15 flagged). But the targeted ML steganalyser, which knows the embedding family, detects every AIDE variant at **100% with AUC 0.998-1.000**. The reason is structural: pair QIM forces $|c_i - c_j| \geq \delta$, leaving a gap natural photos do not have. This is the main item in future scope.

Project objectives: status

Objective	Target	Status
O1 Lightweight model	< 5 ms/block on CPU	Met: 0.16 ms/block, 13 KB model
O2 Adaptive payload	Per-block map, unique per image	Met
O3 Encryption	AES-256-GCM, image-bound key	Met, hardened with scrypt
O4 Robustness	> 98% at JPEG Q50 and noise s=20	Met by 'robust' (100%)
O5 Tamper localization	Per-block hash map + statistics	Met
O6 Recovery	> 98% under moderate attacks	Met by 'robust' (15/16 at 100%)
O7 vs CNN baselines	>= 4x faster, < 10 MB	Partial: fast + tiny, not yet compared
Image quality	PSNR > 42 dB, SSIM > 0.98	Met by 'quality' (43.4 dB / 0.984)

5. Future scope

Direction	What it means
Beat targeted steganalysis	Side-informed embedding with syndrome-trellis codes (STC) or keyed, dithered pair selection, so the DCT pair-gap fingerprint the ML detector finds disappears.
Survive geometric attacks	Rotation, real cropping and aspect changes break the block grid. Add a registration template or embed in an invariant domain (e.g. log-polar / Fourier-Mellin).
Scale the training data	Retrain on ALASKA2 (80k+ images) and several camera sources so both models generalise beyond BOSSbase.
Colour-aware pipeline	Embed in and model chroma channels, including JPEG chroma subsampling, instead of luma only.
Benchmark against deep models	Run the SteganoGAN / HiDDeN comparison (objective O7) and measure on a Raspberry Pi for edge deployment.
Wider media and deployment	Video frames and audio carriers, a mobile client, and a hardened multi-user server with audit logging.

Current limitations to keep in mind

- **quality** profile does not survive noise (0% at $\sigma \geq 10$) by design; use balanced or stronger for noisy channels.
- Messages close to a cover's capacity are less robust than the 156-byte benchmark runs; `aide capacity` reports both limits.
- The ML detector is trained on one camera source; its out-of-distribution guard only softens verdicts on unfamiliar images.

Quality and verification

Every number in this document is measured by code in the repository: 146 automated tests (unit, ground-truth, end-to-end CLI/HTTP, attack recovery), lint, and CI on Python 3.10-3.12 with a Docker build and health check. The full benchmark is in `reports/benchmark/`.

> **Summary.** AIDE delivers adaptive, encrypted, error-corrected image steganography that runs on a plain CPU, meets the report's robustness and recovery targets with the robust profile, and is honest about its one big open problem: resistance to targeted ML steganalysis.